

NLP 243 – Homework 2

Jose Alvarez
jalva182@ucsc.edu

Nov 7, 2025

1. Language Modeling / Next Token Prediction

A language model estimates the probability of a sequence of words. In next-token prediction, the model learns to predict the most likely next token given all previous tokens, capturing dependencies across time. During training, the model minimizes the cross-entropy loss between predictions and true tokens, ensuring that it assigns higher probability to real text sequences.

2. Self-Attention Mechanism

Self-attention allows each token to look at other tokens in the same sequence to gather contextual information. For each token, query, key, and value vectors are computed, and attention weights determine how much information to pull from other positions. This helps the model capture long-range dependencies without fixed-size context windows.

3. Model Architecture and Implementation

The model is a simple decoder-only language model trained for next-token prediction. Its main components are:

- **Embedding Layer:** Maps each input token ID to a dense vector of dimension $d_{model} = 256$ using `nn.Embedding`.
- **Positional Encoding:** Implements sinusoidal positional encodings as in the original Transformer paper to provide token order information.
- **EmbeddingWithPositions Module:** Combines token and positional embeddings into one module.
- **Language Model Head:** A single linear layer projects embeddings to the vocabulary size for next-token prediction.
- **Loss Function:** Uses `CrossEntropyLoss` with `ignore_index = -100` to ignore padding tokens.
- **Optimization:** Adam optimizer (`lr = 3e-4`) with gradient clipping (`max_norm = 1.0`) for stability.

4. Model Summary

Table 1: Model Summary

Component	Description
Token embedding	256-dimensional vectors
Positional encoding	Sinusoidal (fixed)
Dropout	0.1
Output head	Linear layer to vocab size
Parameters	~15M
Optimizer	Adam (lr = 3e-4)
Loss	CrossEntropy (ignore_index = -100)
Task	Next-token prediction
Dataset	Penn Treebank

5. Results and Perplexity

Perplexity decreased steadily across epochs, indicating effective learning. The final test perplexity (≈ 101) aligns with expectations for an embedding-only model.

Table 2: Final Model Performance (Test 3)

Split	Loss	Perplexity
Train	4.58	97.84
Validation	4.79	121.30
Test	4.83	101.71

Earlier runs (Test 1 and Test 2) showed unstable loss and extremely high perplexity due to incorrect normalization in the loss computation. After correcting this issue, the model converged successfully, achieving stable results in Test 3.

Table 3: Training Progress (Test 3)

Epoch	Train Loss	Train PPL	Val Loss	Val PPL
1	15.85	7,619,375.53	9.54	13,876.38
2	9.27	10,621.20	7.27	1,438.41
3	7.05	1,149.78	6.14	462.94
4	5.91	369.97	5.56	258.71
5	5.33	207.27	5.24	189.32
6	5.02	151.74	5.07	159.05
7	4.84	126.43	4.96	142.38
8	4.72	112.52	4.89	132.76
9	4.64	103.76	4.83	124.96
10	4.58	97.84	4.80	121.30

References

- Vaswani et al., *Attention Is All You Need*, 2017.

- Karpathy, A., *minGPT Tutorial*.
- Hugging Face Transformers Documentation.
- Mielke, S. J., *Comparing Perplexities*, 2019.